SHERLOCK

Security Review For Sodax

Collaborative Audit Prepared For:	**Sodax**
Lead Security Expert(s):	**cybasecurity**
Date Audited:	**February 9 - February 18, 2026**

Introduction

The audit covers the core Stacks (Clarity) smart contracts that form the foundation of the SodaX Protocol on the Stacks network. This includes the Relay Connection contracts, which manage cross-chain messaging and execution; the Spoke Asset Manager contracts, which handle asset custody and cross-chain asset operations on the Stacks spoke chain; and the Rate Limiter contract, which enforces transaction and withdrawal limits to mitigate systemic and liquidity risks. Together, these components enable secure cross-chain interoperability while maintaining operational safeguards on the Stacks environment.

Scope

Repository: icon-project/intent-relay

Audited Commit: 57341f2dc5b24c0d1d3bf4111caf7a89616e9094

Final Commit: 69d635c23b0f1a56b71cdc420cc234e48e2368e6

Files:

- contracts/stacks/contracts/connection/connection-trait.clar
- contracts/stacks/contracts/connection/connection-v3.clar
- contracts/stacks/contracts/connection/connection-v3-state.clar

---

Repository: icon-project/sodax-contracts

Audited Commit: b6a2be2dec6d5062b5a98285cebf2b83eadf2700

Final Commit: c943f808a9b06a13973fbf4e131b5b82811cb305

Files:

- clarity/contracts/asset-manager/asset-manager-impl.clar
- clarity/contracts/asset-manager/asset-manager-state.clar
- clarity/contracts/rate-limit/rate-limit-impl.clar
- clarity/contracts/rate-limit/rate-limit-state.clar
- clarity/contracts/rate-limit/rate-limit-trait.clar
- clarity/contracts/tokens/bnusd.clar

Findings

Each issue has an assigned severity:

- High issues are directly exploitable security vulnerabilities that need to be fixed.

- Medium issues are security vulnerabilities that may not be directly exploitable or may require certain conditions in order to be exploited. All major issues should be addressed.
- Low/Info issues are non-exploitable, informational findings that do not pose a security risk or impact the system's integrity. These issues are typically cosmetic or related to compliance requirements, and are not considered a priority for remediation.

Issues Found

High	Medium	Low/Info
4	6	21

Issues Not Fixed and Not Acknowledged

High	Medium	Low/Info
0	0	0

Issue H-1: Stacks Asset Manager Can Be Drained Via Malicious Token [RESOLVED]

Source: https://github.com/sherlock-audit/2026-02-sodax-clarity-feb-9th/issues/12

Summary

The `asset-manager-state` contract can be drained via a malicious token. The attack is possible due to the permissionless nature of the Sodax bridge architecture, combined with Stacks' `as-contract` context propagation mechanism.

Vulnerability Detail

The Stacks asset manager accepts any contract that implements SIP-10 functionality. An attacker can deploy a malicious SIP-10 compliant token on Stacks and route it through the full cross-chain flow to obtain a valid, validator-signed withdrawal message, then use that message to execute arbitrary transfers from the Clarity asset manager (or token bank).

Example steps to fulfill the attack:

Step 1: Deploy a malicious SIP-10 token on Stacks

The attacker deploys a malicious contract implementing the SIP-10 trait whose `transfer` function contains drain logic but only if it's activated, otherwise it behaves similar to any normal SIP-10 token.

Attacker also self-mints a large amount of tokens to himself.

Step 2: Register the malicious token on the EVM hub

The attacker then calls AssetManager::deployAsset with the Stacks chain ID and malicious token principal bytes, on the hub. The `deployAsset` function is permissionless, as such anyone can call it for any `(chainID, addr)` pair as long as the chain is configured. This creates a hub-side `AssetToken` mapped to the malicious token spoke address, allowing deposits to flow from the Stacks spoke.

Step 3: Initiate a deposit on Stacks

The attacker calls `asset-manager-impl::transfer` with the malicious token principal and his EVM EOA address as the recipient. Since the malicious token is still dormant, the transfer call works as intended. Note, since the `malicious-token::transfer` call is done outside the context of `as-contract`, in the `asset-manager-state::deposit` function, no funds could be stolen at this point regardless.

Step 4: Round-trip through the hub

The deposit message arrives at the hub via the `AssetManager::recvMessage` function and is processed. The hub's `_processMessage` looks up the `AssetToken` registered in step 2 and

mints hub tokens to the attacker.

With a balance of hub tokens, the attacker now calls `AssetManager::transfer` on the EVM hub, which burns the hub tokens and sends a signed withdrawal message back to the Stacks spoke. The withdrawal payload contains the malicious token's Stacks principal, the attacker Stacks principal as recipient, and the deposited amount.

Step 5: Activate malicious mode

The attacker can now set the token to malicious mode, ensuring that on the next `SIP-10::transfer` call, all caller funds are stolen.

Step 6: Relay message and drain Stacks asset manager

The attacker calls `asset-manager-impl::recv-message` on Stacks with the signed payload and the malicious token trait. All validation checks pass since the attacker passes all valid required input. Specifically, the following checks all pass:

- Connection/rate-limit contract verification: attacker supplies the real contracts
- Signature verification: message is genuinely signed by hub validators
- Source chain/address: message originated from the hub asset manager
- get-token-balance: balance exists, since the asset manager holds the tokens deposited in Step 3
- Amount validation: withdrawal amount matches the deposited amount
- Rate limit::verify-withdraw: unconfigured tokens default to a 0 `rate-per-second` value, which auto-returns `(ok true)`, effectively bypassing rate limiting
- Token payload-trait verification: payload contains the malicious token's address, matching contract of the passed trait

Execution then reaches asset-manager-state::withdraw function:

Here, the as-contract sets `tx-sender` to `asset-manager-state` for the immediate contract call, and this value persists as `tx-sender` within the callee's execution context. As a result, the malicious token's `transfer` function can execute functionality to steal all tokens from the `tx-sender` (asset manager).

In the example malicious behavior, the stx-transfer? executes successfully because it requires `sender` (second argument) to be equal to `tx-sender`, which they both are `asset-manager-state`. The entire STX balance of the asset manager is transferred to the attacker.

The same technique can be extended to steal any SIP-10 token held by the asset manager by nesting a `contract-call?` to the legitimate token's `transfer` function, since `tx-sender` remains `asset-manager-state` throughout the call chain.

Impact

Complete loss of all funds held by the asset manager.

Code Snippet

asset-manager-state.clar#L80

Tool Used

Manual Review

Recommendation

Before Clarity4 (SIP-33), it was not possible to safely allow arbitrary tokens to be processed by a system where one contract holds multiple token balances. The developer would either:

1. Create and deploy vaults for each token, so that the balances are separate
2. Add a whitelist mechanism and vet each token, so that no malicious tokens are added.

For Sodax to keep the current design and not use any of the above methods (which would require extensive refactoring), the new as-contract? asset restrictions, added with Clarity4, can be used.

Example implementation for the `asset-manager-state::withdraw` function, which will ensure that when the `SIP-10::transfer` function is called, only outflows of that specific token are allowed from the asset manager contract.

Notes:

- Changing to Clarity4 will bring several other benefits, which will be detailed in a different finding
- Instead of `tokens-bank`, the new Clarity4 current-contract keyword can be used to express the same thing

Discussion

gcranju

Fixed with migration to clarity v4
https://github.com/icon-project/sodax-contracts/pull/592

abarbatei

Confirmed, the issue was fixed in the mentioned PR.

Issue H-2: Anyone Can Pause The System [RESOLVED]

Source: https://github.com/sherlock-audit/2026-02-sodax-clarity-feb-9th/issues/14

Summary

The pause functionality can be triggered by any user because the connection contract provided as input is not validated.

Vulnerability Detail

In `rate-limit-impl::pause`, the connection parameter is supplied directly by the caller and is not verified to ensure that it is the legitimate connection contract implementation.

This connection parameter is passed to `decode-and-validate-payload`, where it is used to recover the public key from the payload signature. The recovered public key must correspond to a hub admin or signer.

However, since the contract does not validate the authenticity of the provided connection, a malicious user can supply a fake connection contract. In such a contract, `fake-connection::recover-public-key` can be implemented to always return the hub admin's public key, regardless of the message hash or signature provided.

As a result, signature validation can be bypassed.

Impact

Any user can successfully call the `pause` function.

Code Snippet

rate-limit-impl.clar#L176 rate-limit-impl.clar#L245

Tool Used

Manual Review

Recommendation

Within the `pause` function, validate that the supplied connection contract matches the trusted implementation stored in `rate-limit-state::get-connection-impl` before proceeding with signature verification.

Discussion

gcranju

Acknowledged and Fixed https://github.com/icon-project/sodax-contracts/pull/593

abarbatei

Confirmed, the issue was fixed in the mentioned PR.

Issue H-3: Missing Unpause Action [RESOLVED]

Source: https://github.com/sherlock-audit/2026-02-sodax-clarity-feb-9th/issues/15

Summary

The `rate-limit-impl` contract does not implement any mechanism to unpause the system once it has been paused.

Vulnerability Detail

The `rate-limit-state` contract includes an `unpause` function that can only be executed by the `rate-limit-impl` contract:

```
(define-public (unpause)
  (begin
@>  (try! (is-rate-limit))
    (ok (var-set paused false))
  )
)
```

The `is-rate-limit` check ensures that only the authorized `rate-limit-impl` contract can trigger the unpause action.

However, the current `rate-limit-impl` contract does not implement any functionality to call `unpause`. As a result, once withdrawals are paused, there is no mechanism to re-enable them.

Impact

Withdrawals cannot be resumed after being paused, so user funds remain locked until `rate-limit-impl` is updated.

Code Snippet

rate-limit-impl.clar#L176

Tool Used

Manual Review

Recommendation

Implement a corresponding `unpause-action` within the `rate-limit-impl` contract. Ensure that this action is properly handled within `recv-message`, similar to how the `pause-action` functionality is processed.

Discussion

gcranju

Acknowledged and Fixed https://github.com/icon-project/sodax-contracts/pull/593

abarbatei

Confirmed, the issue was fixed in the mentioned PR. The team opted to use a different approach where the admin can directly call the `rate-limit-state::unpause` and `rate-limit-state::pause` functions.

Issue H-4: Missing Auth In Hub Broadcast Enables Malicious Cross Chain Replay [RESOLVED]

Source: https://github.com/sherlock-audit/2026-02-sodax-clarity-feb-9th/issues/35

Summary

The hub admin and signer configuration can be corrupted by submitting improperly formatted data copied from other chain spokes' hub messages.

Vulnerability Detail

The SpokesManager::broadcastMessage function is permissionless and is used to emit hub-side events for hub actions. The caller supplies:

- the payload
- the payload signature
- the destination chain IDs
- the destination addresses

The event is emitted in ConnectionV3::sendMessage as follows:

```
emit Message(
    chainId,
    abi.encodePacked(msg.sender),
    connSn,
    dstChainId,
    dstAddress,
    payload
);
```

Because `msg.sender` in this context is always the `SpokesManager`, the emitted event does not include any information allowing the validators to distinguish between calls made by legitimate operators and those made by a malicious attacker.

As a result, validators will sign hub action events regardless of who invoked `SpokesManager::broadcastMessage`. This behavior is reflected in the off-chain index::parseAndProcessLogs function, which filters only for successful, finalized transactions originating from the legitimate hub connection contract. These filtered events are then forwarded to lambda-utils::signMessage for signing.

Once a sufficient number of validators have signed a message, it can be executed on any spoke.

Cross-Chain Format Mismatch Issue

When updating the hub admin or hub signers, payload formats differ between EVM spokes and the Stacks spoke:

- EVM spokes expect 20-byte addresses
- Stacks spoke expects 33-byte public keys

For example, when updating the hub admin:

- The operator calls `SpokesManager::broadcastMessage` for EVM spokes with a payload containing the new 20-byte address and the corresponding signature.
- The operator then calls `SpokesManager::broadcastMessage` for the Stacks spoke with a different payload and signature, containing the corresponding 33-byte public key format.

Consider the following scenario:

- A legitimate operator calls `SpokesManager::broadcastMessage` for an EVM spoke with the correct payload (20-byte admin), signature, and destination details.
- An attacker copies the same payload and signature and calls `SpokesManager::broadcastMessage` again, but this time specifies the Stacks destination chain and address.
- Two `ConnectionV3` events are emitted: one for the EVM spoke and one for the Stacks spoke, but both contain the same 20-byte payload.
- Off-chain validators sign both events.
- The signed messages are relayed to their respective spokes.

On the EVM spoke, the message behaves as intended. On the Stacks spoke, however, the hub admin is incorrectly set to a 20-byte value instead of the expected 33-byte format.

Since Stacks verifies the hub admin using `secp256k1-recover?`, which returns a 33-byte compressed public key, the `rate-limit-state::is-hub-admin` check will always fail.

The same issue applies to the set hub signers action, and can be performed through SpokesManager::sendMessage as well.

Impact

Hub admin and hub signer protected operations become inaccessible due to the format mismatch.

Code Snippet

SpokesManager.sol:L65 ConnectionV3.sol:L87 rate-limit-impl.clar:L254-L256

Tool Used

Manual Review

Recommendation

To completely block off this attack vector, both for the Stacks spoke and any other potentially vulnerable spoke, introduce access control in `SpokesManager` for the `sendMessage` and `broadcastMessage` functions.

This would then allow only approved operators to send messages, blocking attackers from triggering it.

Also, on the Stacks client side, ensure this case is also treated, so that, by mistake, no such configuration is allowed. For that, check that the admin key is always `33` in size and that it starts with `0x02` or `0x03` (valid compressed secp256k1 prefixes). Also apply these checks for each key belonging to signers.

From an implementation point of view, a solution similar to the following example can be used:

```
@@ -44,6 +44,7 @@
 (define-constant err-paused (err u911))
 (define-constant err-already-executed (err u912))
 (define-constant err-invalid-action (err u913))
+(define-constant err-invalid-key-format (err u914))
 ;;

 ;; -----------------------------------------------------------------------------
@@ -235,6 +236,19 @@
 ;; Private Helpers
 ;; -----------------------------------------------------------------------------

+(define-private (is-valid-compressed-key (key (buff 4096)))
+  (let ((prefix (unwrap! (element-at? key u0) false)))
+    (and
+      (is-eq (len key) u33)
+      (or (is-eq prefix 0x02) (is-eq prefix 0x03))
+    )
+  )
+)
+
+(define-private (check-valid-key (key (buff 4096)) (valid bool))
+  (and valid (is-valid-compressed-key key))
+)
+
 ;; @dev Decodes the hub payload structure:
 ;;      [data (buff), signature (buff 65)]
 (define-private (decode-and-validate-payload (payload (buff 4096)) (connection
 ↪  <connection-contract>))
```

```
@@ -302,6 +316,7 @@
     (unwrap-panic (begin
       (if (is-eq kind hub-admin-set-action)
         (let ((admin (unwrap-panic (element-at? data u0))))
+          (asserts! (is-valid-compressed-key admin) err-invalid-key-format)
           (contract-call? .rate-limit-state set-hub-admin admin)
         )
         (if (is-eq kind hub-signers-set-action)
@@ -309,6 +324,7 @@
               (signers (unwrap-panic (element-at? data u0)))
               (signers-list (contract-call? .rlp-decode rlp-to-list signers))
             )
+            (asserts! (fold check-valid-key signers-list true)
 ↪  err-invalid-key-format)
             (contract-call? .rate-limit-state set-hub-signers signers-list)
           )
           (if (is-eq kind rate-limit-set-action)
```

Discussion

gcranju

Acknowledged and Fixed alongwith issue #28

abarbatei

Confirmed, the issue was fixed in the mentioned PR.

Issue M-1: Token Payer Is Not Counted As Message Sender [RESOLVED]

Source: https://github.com/sherlock-audit/2026-02-sodax-clarity-feb-9th/issues/16

Summary

When an asset transfer is initiated on Stacks, the funds are taken from the `tx-sender` but the `contract-caller` is mistakenly added to the cross-chain message `sender` field.

Because of that, when a Stacks contract calls `transfer` on behalf of its users, the hub derives a wallet using the contract's address instead of the user's.

This breaks the hub's wallet model, as users that pay lose access to all wallet-level actions (`revertStored`, `executeStored`) and refunds are misdirected.

Vulnerability Detail

Initiating an asset transfer on Stacks is done by calling the `asset-manager-impl::transfer` function. In this function, the `contract-caller` is encoded in the `message` variable as the `sender`.

However, when the deposit is processed in the `asset-manager-state::deposit` function, the tokens are taken from the `tx-sender`

This means that, when a user calls `asset-manager-impl::transfer` directly on Stacks (`tx-sender == contract-caller`) everything works correctly.

But when any intermediary contract calls `transfer` on behalf of its users (a DEX aggregator, yield optimizer, or any protocol composing with the bridge), the identities diverge:

- `tx-sender` = user; tokens deducted from the user
- `contract-caller` = integrating contract; cross-chain message records the contract as the sender

Every other spoke implementation in the protocol uses the **same identity** for both the token payment and the cross-chain message sender. Stacks is uniquely broken because its `deposit` function reaches through to `tx-sender` while the message uses `contract-caller`

Example, looking at the EVM SpokeAssetManager::transfer for comparison:

```
IERC20(token).safeTransferFrom(msg.sender, address(this), amount);  // tokens FROM
↪  msg.sender
Transfer memory _transfer = Transfer(token.toBytes(), msg.sender.toBytes(), to,
↪  amount, data);  // sender = msg.sender
```

Coupled with this mismatch, is the issue that on Stacks, a contract can initiate a deposit for its `tx-sender`. Meaning that a malicious contract may initiate an intentionally failing transfer on the Hub (random DeFi call) which will get the funds refunded to the `contract-caller` instead of the original `tx-sender`.

Note, if a user calls a malicious contract, that contract can already drain their funds directly via `tx-sender`, but the last paragraph shows a situation that should not be allowed regardless.

Impact

Stacks spoke implementation is the only one in the system where the deposit identity and the message sender identity can diverge, breaking protocol invariants.

Also on the Stacks spoke, calling contracts can force a transfer from original senders, which should not be allowed.

Code Snippet

sodax-contracts/clarity/contracts/asset-manager/asset-manager-impl.clar:L103

sodax-contracts/clarity/contracts/asset-manager/asset-manager-state.clar:L70-L71

Tool Used

Manual Review

Recommendation

Add a guard to block indirect calls in the `asset-manager-impl::transfer` function, ensuring that the `contract-caller` is always equal to the `tx-sender`:

The change has no effect on direct user calls where `tx-sender == contract-caller` already holds.

For integrating contracts, it requires them to first pull tokens from the user into their own balance, then bridge via an `as-contract` call making the contract both the `tx-sender` and the `contract-caller`. This preserves the same-identity invariant across the deposit and the cross-chain message, and prevents contracts from arbitrarily initiating transfers on behalf of the original transaction signer.

Discussion

gcranju

Acknowledged and Fixed https://github.com/icon-project/sodax-contracts/pull/594

abarbatei

Confirmed, the issue was fixed in the mentioned PR.

The team opted to use a different approach where `asset-manager-state::deposit` function now accepts a `sender` parameter, and the `asset-manager-impl::transfer` function calls that function with the `contract-caller`. This forces an implicit condition where, only if the `contract-caller` is the `tx-sender`, will pulling tokens via the `deposit` function work.

Note: with this implementation, integrators who don't call the contract as intended will receive the `u4: sender is not the same as tx-sender` error code.

Issue M-2: Withdrawal DoS via Rate Limit Griefing [ACKNOWLEDGED]

Source: https://github.com/sherlock-audit/2026-02-sodax-clarity-feb-9th/issues/22

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

Withdrawals can be temporarily denied through a griefing attack in which an attacker repeatedly performs cross-chain transfers to deliberately exhaust the available rate limit for a token.

Vulnerability Detail

During a withdrawal in `asset-manager-impl::recv-message`, the function `rate-limit-impl::verify-withdraw` is invoked to deduct the withdrawn amount from the token's available rate limit.

The rate limit mechanism works as follows:

- The available limit is linearly replenished based on the token's configured replenishment rate and time elapsed since the last update.
- The requested withdrawal amount is then subtracted from the updated available limit.

Because the available limit increases solely as a function of elapsed time, an attacker can perform rapid cross chain transfers to consume the available capacity.

As a result, the token's available limit can be driven to zero, preventing legitimate users from bridging that token until enough time passes for the limit to replenish to a reasonable amount.

Impact

Temporary DoS for withdrawals of the affected token(s).

Code Snippet

https://github.com/sherlock-audit/2026-02-sodax-clarity-feb-9th/blob/main/sodax-contracts/clarity/contracts/rate-limit/rate-limit-impl.clar#L110
https://github.com/sherlock-audit/2026-02-sodax-clarity-feb-9th/blob/main/sodax-contracts/clarity/contracts/rate-limit/rate-limit-impl.clar#L283

Tool Used

Manual Review

Recommendation

In addition to the existing time-based replenishing rate limit up to a maximum value, introduce a secondary temporary limit with the following properties:

- It accumulates recent deposits.
- It decays gradually over time.

The effective rate limit would then be calculated as the sum of the standard replenishing limit and the decaying temporary limit.

On withdrawals, substract the withdrawn amount from the decaying limit, and if any excess amount is present, substract it from the replenishing limit.

This approach ensures that if a deposit is quickly withdrawn on the same chain, it does not meaningfully reduce the token's usable rate limit, and if it's not, its impact on the rate limit gradually diminishes over time. Such a mechanism mitigates griefing while preserving the intended rate limiting behavior.

Note: an example Clarity implementation can be seen in the Granite project, where they implemented daily withdrawal caps, and faced the same issue. Their decay withdraw implementation can be seen here.

Discussion

gcranju

Acknowledged. This behavior is an inherent property of a global per-token rate limit and is an accepted design tradeoff. We evaluated more complex mitigations (e.g., dual-bucket or decaying limits) but found they introduce additional attack surface and accounting complexity. Instead, rate limits are additionally monitored and enforced by an off-chain backend, which can safely restore capacity when exhaustion occurs due to valid traffic.

Issue M-3: Incorrect Hub Signers Configuration [RESOLVED]

Source: https://github.com/sherlock-audit/2026-02-sodax-clarity-feb-9th/issues/23

Summary

The `hub-signers-set-action` command, which is executed on the `rate-limit-impl` contract, does not function correctly due to improper decoding logic in `rlp-decode::to-list`.

Vulnerability Detail

In `rate-limit-impl::process-hub-message`, when a `hub-signers-set-action` message is processed, the list of hub signers is decoded as follows:

```
(if (is-eq kind hub-signers-set-action)
  (let (
      (signers (unwrap-panic (element-at? data u0)))
      (signers-list (contract-call? .rlp-decode rlp-to-list signers))
    )
    (contract-call? .rate-limit-state set-hub-signers signers-list)
  )
```

The function `rlp-decode::rlp-to-list` internally calls `rlp-decode::to-list`. While this function correctly decodes the first eight elements of the RLP list, it fails when decoding the ninth element.

Below is the relevant portion of the decoding logic:

```
(if (is-eq  d8_ 0x)
  (list i1 i2 i3 i4 i5 i6 i7)
  (let (
    (d8 (decode-item  d8_))
    (i8 (get-item  d8))
    (d9_ (get-rlp  d8))
  )
@>(if (is-eq  d8_ 0x)
    (list i1 i2 i3 i4 i5 i6 i7 i8)
    (let (
@>    (d9 (decode-item  d8_))
@>    (i9 (get-item  d8))
    )
    (list i1 i2 i3 i4 i5 i6 i7 i8 i9)
    ))
```

The issue is that the implementation mistakenly reuses `d8_` and `d8` when attempting to decode the ninth element, instead of using `d9_` and `d9`. As a result, it re-decodes the eighth element rather than properly decoding the ninth.

Additionally, the function never attempts to decode more than nine elements, even though the system supports up to ten hub signers. The tenth hub signer, if present, would be silently ignored.

Impact

When the hub admin configures a 9th or a 10th signer, they will not be registered as valid hub signers. Configuration messages they sign will therefore be rejected.

Code Snippet

rlp-decode.clar#L179-L186

Tool Used

Manual Review

Recommendation

Update `rlp-decode::to-list` to correctly decode the 9th element, and extend it to support up to 10 elements.

Example implementation:

```
-  (if (is-eq  d8_ 0x)
+  (if (is-eq  d9_ 0x)
     (list i1 i2 i3 i4 i5 i6 i7 i8)
     (let (
-      (d9 (decode-item  d8_))
-      (i9 (get-item  d8))
+      (d9 (decode-item  d9_))
+      (i9 (get-item  d9))
+      (d10_ (get-rlp  d9))
     )
-    (list i1 i2 i3 i4 i5 i6 i7 i8 i9)
+    (if (is-eq  d10_ 0x)
+      (list i1 i2 i3 i4 i5 i6 i7 i8 i9)
+      (let (
+        (d10 (decode-item  d10_))
+        (i10 (get-item  d10))
+      )
+      (list i1 i2 i3 i4 i5 i6 i7 i8 i9 i10)
```

```
+       )
+     )
```

Additionally, consider using a more claritonic pattern for readability, for example using unwrap! for early exit in case of condition read (similar to how Pyth oracle parses feeds on Stacks):

```
(define-private (try-decode (input (buff 4096)))
  (if (is-eq input 0x)
    none
    (let ((pair (decode-item input)))
      (some { item: (get-item pair), rest: (get-rlp pair) })
    )
  )
)

(define-read-only (to-list (input (buff 4096)))
  (let (
    (d1  (unwrap! (try-decode input)         (list)))
    (i1  (get item d1))
    (d2  (unwrap! (try-decode (get rest d1)) (list i1)))
    (i2  (get item d2))
    (d3  (unwrap! (try-decode (get rest d2)) (list i1 i2)))
    (i3  (get item d3))
    (d4  (unwrap! (try-decode (get rest d3)) (list i1 i2 i3)))
    (i4  (get item d4))
    (d5  (unwrap! (try-decode (get rest d4)) (list i1 i2 i3 i4)))
    (i5  (get item d5))
    (d6  (unwrap! (try-decode (get rest d5)) (list i1 i2 i3 i4 i5)))
    (i6  (get item d6))
    (d7  (unwrap! (try-decode (get rest d6)) (list i1 i2 i3 i4 i5 i6)))
    (i7  (get item d7))
    (d8  (unwrap! (try-decode (get rest d7)) (list i1 i2 i3 i4 i5 i6 i7)))
    (i8  (get item d8))
    (d9  (unwrap! (try-decode (get rest d8)) (list i1 i2 i3 i4 i5 i6 i7 i8)))
    (i9  (get item d9))
    (d10 (unwrap! (try-decode (get rest d9)) (list i1 i2 i3 i4 i5 i6 i7 i8 i9)))
    (i10 (get item d10))
  )
    (list i1 i2 i3 i4 i5 i6 i7 i8 i9 i10)
  )
)
```

Discussion

gcranju

Acknowledged and Fixed: https://github.com/icon-project/sodax-contracts/pull/596

abarbatei

Confirmed, the issue was fixed in the mentioned PR.

Issue M-4: Incorrect Access Control For Hub Actions [RESOLVED]

Source: https://github.com/sherlock-audit/2026-02-sodax-clarity-feb-9th/issues/24

Summary

The access control logic for hub actions is flawed. As currently implemented, it allows regular signers to execute actions that should be restricted to admins only. The issues was found by the development team during the audit.

Vulnerability Detail

The current access control logic for hub actions is implemented as follows:

```
(if (or (is-eq kind rate-limit-set-action) (is-eq kind hub-signers-set-action))
  (asserts! (contract-call? .rate-limit-state is-hub-admin public-key)
  ↪  err-not-hub-admin)
  (asserts! (contract-call? .rate-limit-state is-hub-admin-or-signer public-key)
  ↪  err-not-hub-admin-signers)
)
```

This logic is incorrect because it enables regular signers to update the admin. A signer can assign themselves as the new admin and thereby obtain privileges to execute any admin-only actions.

Instead, updating the admin should be strictly restricted to the current admin.

Furthermore, for consistency with the EVM implementation, setting the rate limit should be permitted for both admins and signers.

Impact

A signer can escalate priviledges by setting themselves as the admin. Additionally, signers are restricted from performing rate limit updates.

Code Snippet

rate-limit-impl.clar#L254-L256

Tool Used

Manual Review

Recommendation

Modify the conditional logic in `rate-limit-impl` as follows:

```
-(if (or (is-eq kind rate-limit-set-action) (is-eq kind hub-signers-set-action))
+(if (or (is-eq kind hub-admin-set-action) (is-eq kind hub-signers-set-action))
  (asserts! (contract-call? .rate-limit-state is-hub-admin public-key)
  ↪  err-not-hub-admin)
  (asserts! (contract-call? .rate-limit-state is-hub-admin-or-signer public-key)
  ↪  err-not-hub-admin-signers)
)
```

Discussion

gcranju

Acknowledged, and already updated with token malicious update

https://github.com/icon-project/sodax-contracts/pull/592/changes allowbreak #diff-98a60df118653138d7161aedd84122f4f65dc4e2d76dc21016e4012e2683b9dbR254

abarbatei

Confirmed, the issue was fixed in the mentioned PR and identified by the team during the audit, as mentioned in the summary.

Issue M-5: State Storage Mixing Can Cause Unexpected Configurations [ACKNOWLEDGED]

Source: https://github.com/sherlock-audit/2026-02-sodax-clarity-feb-9th/issues/25

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

Hub messages may be replayed or incorrectly interpreted because part of the rate limit state is stored within the `rate-limit-impl` contract.

Vulnerability Detail

The `executed-messages` map functions as replay protection for hub messages by storing the deadline of each processed message.

However, since this map is defined in the `rate-limit-impl` contract, it does not persist across updates of the implementation.

As a result, when the `rate-limit` implementation is upgraded, previously stored processing deadlines are lost. This allows already processed but not yet expired hub messages to be replayed.

This issue is particularly important for the pause action, as it is the only action that bypasses the nonce based replay protection at the connection layer.

Additionally, the action constants are also defined within the implementation contract:

```
(define-constant hub-admin-set-action u1)
(define-constant hub-signers-set-action u2)
(define-constant rate-limit-set-action u3)
(define-constant rate-limit-reset-action u4)
(define-constant pause-action u5)
```

If these constant values are modified in a future implementation version, any pending hub messages that have not yet been processed could be interpreted as a different action than originally intended, resulting in unexpected configurations.

Impact

Following an update to `rate-limit-impl`, hub messages may be replayed or misinterpreted, leading to unintended system behavior.

Code Snippet

https://github.com/sherlock-audit/2026-02-sodax-clarity-feb-9th/blob/main/sodax-contracts/clarity/contracts/rate-limit/rate-limit-impl.clar#L24-L28
https://github.com/sherlock-audit/2026-02-sodax-clarity-feb-9th/blob/main/sodax-contracts/clarity/contracts/rate-limit/rate-limit-impl.clar#L52

Tool Used

Manual Review

Recommendation

Relocate the `executed-messages` map and the action constants to the `rate-limit-state` contract so that their values persist across implementation upgrades and maintain consistent message interpretation.

Discussion

gcranju

Acknowledged. The executedMessages map is a secondary replay check limited to a short, fixed deadline window (currently 1 hour), so persisting it across upgrades was not deemed necessary. Action constants are part of the implementation logic rather than protocol data and are intentionally defined in the implementation contract and managed through the upgrade process id any changes is decided.

Issue M-6: bnUSD Is Not SIP-10 Compliant [RESOLVED]

Source: https://github.com/sherlock-audit/2026-02-sodax-clarity-feb-9th/issues/32

Summary

The `bnusd.clar` and `soda.clar` tokens are not SIP-010 compliant. Their `transfer` function returns `(err u101)` when `tx-sender` does not match `sender`, but the SIP-010::transfer requires `(err u4)` to be returned in this case.

Vulnerability Detail

The SIP-010 fungible token standard defines specific error codes for the transfer function:

error code	reason
u1	`sender` does not have enough balance
u2	`sender` and `recipient` are the same principal
u3	`amount` is non-positive
u4	`sender` is not the same as `tx-sender`

> When returning an error in this function, the error codes should follow the same patterns as the built-in ft-transfer? and stx-transfer? functions.

Both `bnusd.clar` and `soda.clar` use `err-not-token-owner (err u101)` for the sender authorization check instead of the required `(err u4)`:

Impact

Off-chain tools, wallets and any SIP-010-aware contract or indexer that checks for the standard `u4` error code to detect unauthorized transfers will not correctly identify this failure. This breaks interoperability with the broader Stacks ecosystem that relies on standard error codes for programmatic error handling.

Code Snippet

- bnusd.clar#L3-L12
- soda.clar#L3-L12

Tool Used

Manual Review

Recommendation

Replace `err-not-token-owner` with the SIP-010 standard error code `u4` in both token contracts:

Discussion

gcranju

Already fixed in #29

abarbatei

Confirmed, the issue was fixed in the mentioned PR.

Issue L-1: Possible bnUSD External Integration Issues Due To Missing Contract Caller Authorization [RESOLVED]

Source: https://github.com/sherlock-audit/2026-02-sodax-clarity-feb-9th/issues/10

Summary

With regards to `SIP-10:transfer` token authorization, the Stacks DeFi ecosystem has evolved, over time, from a `tx-sender` only authorization to an either `tx-sender` or `contract-caller` authorization model.

The `bnUSD` token does not follow this new authorization approach and may create issues for third party integrators.

Vulnerability Detail

In the `bnusd::transfer` function, the token transfer authorization is done by checking that the `sender` is the same as the `tx-sender`:

More modern tokens have started to permit the sender to be either the `tx-sender` or the `contract-caller`, to allow for token-bearing contracts to more easily transfer assets without relying on `as-contract`.

Over time, new SIPs that worked with token transfers have specifically recommended this behavior, e.g., SIP-13:transfer:

> Since it is possible for smart contracts to own tokens, it is recommended to check for both `tx-sender` and `contract-caller` as it allows smart contracts to transfer tokens it owns without having to resort to using `as-contract`. Such a guard can be constructed as follows

Note, the chain native-BTC wrapper, `sBTC`, also implements this behavior.

Impact

Integration issues with third party protocols.

Code Snippet

https://github.com/sherlock-audit/2026-02-sodax-clarity-feb-9th/blob/main/sodax-contracts/clarity/contracts/tokens/bnusd.clar#L12

Tool Used

Manual Review

Recommendation

Change the `(asserts! (is-eq tx-sender sender) err-not-token-owner)` check in the `bnusd::transfer` function to also allow for contract caller authorization.

Example implementation:

```
- (asserts! (is-eq tx-sender sender) err-not-token-owner)
+ (asserts! (or (is-eq tx-sender sender) (is-eq contract-caller sender))
 ↪  err-not-token-owner)
```

Discussion

gcranju

Already fixed in #11

abarbatei

@gcranju the `SIP-10::transfer` function is a special case where you need both `tx-sender` and `contract-caller` authorization. The fix you mentioned changed the logic to a `contract-caller` check only but you need a **or** `tx-sender`, **or** `contract-caller`.

Please apply the mentioned recommendation

gcranju

Updated in issue #11

abarbatei

Confirmed that the issue was fixed in the mentioned PR following the update.

Issue L-2: Do Not Use tx-sender For Caller Authorization [RESOLVED]

Source: https://github.com/sherlock-audit/2026-02-sodax-clarity-feb-9th/issues/11

Summary

Using `tx-sender` instead of `contract-caller` for specific (non-transfer) authorization checks, leaves users vulnerable to abuse in case they fall prey to phishing attacks.

Vulnerability Detail

In the `bnusd` contract there are instances where `tx-sender` is used instead of `contract-caller` for caller authorization.

By doing this, if the owner falls to phishing scams and interacts with any malicious contract, that malicious contract can then interact with the `bnusd` contract and execute sensitive operations.

For example, the `bnUSD` contract owner may interact with a malicious contract and that malicious contract would then be able to mint infinite tokens via the `bnusd::mint` function.

Note, this situation requires user error, however it should not be permitted either way.

Impact

Phished bnUSD owner, while interacting with random malicious contracts, will allow minting of infinite bnUSD tokens.

Code Snippet

https://github.com/sherlock-audit/2026-02-sodax-clarity-feb-9th/blob/main/sodax-contracts/clarity/contracts/tokens/bnusd.clar#L21

https://github.com/sherlock-audit/2026-02-sodax-clarity-feb-9th/blob/main/sodax-contracts/clarity/contracts/tokens/bnusd.clar#L53

Tool Used

Manual Review

Recommendation

Use `contract-caller` instead of `tx-sender` in the `bnusd::transfer-contract-ownership` and `bnusd::mint` functions.

Discussion

gcranju

Acknowledged and Fixed https://github.com/icon-project/sodax-contracts/pull/595

abarbatei

Confirming the fix is correct and resolves this current finding.

However, please keep in mind the change to `transfer` is not enough to resolve issue #10. More information is mentioned in the issue page.

Issue L-3: Signature Threshold Configuration Issues [RESOLVED]

Source: https://github.com/sherlock-audit/2026-02-sodax-clarity-feb-9th/issues/13

Summary

The signature threshold for bridging messages can be configured to up to 50 signers on Stacks, but only up to 10 signers can be verified, resulting in a temporary DoS for message processing whenever a threshold higher than 10 signers is configured.

Also, the EVM equivalent implementation allows for up to 99 signers, while the Stacks implementation allows only up to 50. This may create a mismatched validation threshold on the system globally.

Vulnerability Detail

The validator set and signature threshold can be updated via `connection-v3::set-validators` and `connection-v3::set-validators-threshold`, allowing up to `50` signers and a threshold of up to `50`.

However, there are instances in the codebase where only 10 signatures are allowed:

In `connection-v3::verify-signature`, only up to 10 unique accumulated, valid, signatures are allowed:

Also, the `rate-limit-impl::recv-message` function only supports a maximum of `10` signers for the `signatures` param:

As a result, whenever the system is configured to use a threshold above `10`, messages cannot be successfully verified.

Another key difference is that, on EVM implementations, the maximum allowed number of validators is up to 99:

- ConnectionV3.sol#L10
- ConnectionV3.sol#L53

```
uint256 constant MAX_VALIDATORS = 100;

// ...

function updateValidators(
    bytes[] memory _validators,
    uint8 _threshold
) external onlyOwner {
```

```
    require(_threshold > 0, "Invalid threshold");
    require(validators.length < MAX_VALIDATORS, "Invalid Amount of validators");
```

Impact

Configuring the signature threshold between 11 and 50 causes message processing to halt.

Also, if Sodax decides to add over 50 signers and threshold for the system, globally, the Stacks component cannot support it and would need to be less secure and have a custom threshold.

Code Snippet

intent-relay/contracts/stacks/contracts/connection/connection-v3.clar:L142

intent-relay/contracts/stacks/contracts/connection/connection-v3.clar:L156

sodax-contracts/clarity/contracts/rate-limit/rate-limit-impl.clar:L137

Tool Used

Manual Review

Recommendation

In case the 50 validators limit is intended for the Stacks client, update `connection-v3::verify-signature` to support up to `50` signatures to match the maximum configurable signers count.

Example changes in `verify-signature`:

```
- unique-validators-list: (list 10 (buff 100))}))
+ unique-validators-list: (list 50 (buff 100))}))


- (append (get unique-validators-list accumulator) recovered-key) u10))}))
+ (append (get unique-validators-list accumulator) recovered-key) u50))}))
```

Also allow in `rate-limit-impl::recv-message`, up to 50 signatures:

```
(define-public (recv-message
    (src-chain-id uint)
    (src-address (buff 256))
    (conn-sn uint)
    (payload (buff 4096))
-   (signatures (list 10 (buff 65)))
```

```
+   (signatures (list 50 (buff 65)))
    (connection <connection-contract>)
  )
```

However, if the 99 limit is intended, then further change all code instances to cap the list from 50 to 99.

Discussion

gcranju

Acknowledged: We won't be using more than 10 for now, so, this would be fine!

abarbatei

Since the team is aware and does not plan to increase the number of signers to above 10, then we'll downgrade the finding severity to a low/info, to serve as a note that there is a discrepancy between 50 vs 10 limit implementation.

gcranju

Hey @abarbatei , even though we do not plan to exceed 10 signers at this time, we have updated the implementation to match the documented limit to avoid inconsistencies going forward.

https://github.com/icon-project/sodax-contracts/pull/603
https://github.com/icon-project/intent-relay/pull/372

abarbatei

@gcranju, the two PRs cover all existing instances for this commit and fix them, confirming.

Please do note, that when the fix for issue #18 will be merged, the `(unique-validators-list (list 10 (buff 100))))` also needs to be changed to a `(list 50` also.

Issue L-4: Validator Threshold Should Never Be Zero [RESOLVED]

Source: https://github.com/sherlock-audit/2026-02-sodax-clarity-feb-9th/issues/17

Summary

The validator threshold is allowed to be set to 0. This permits a misconfiguration of no validation needed for messages, which would allow arbitrary message execution on Stacks.

Vulnerability Detail

In the `connection-v3::set-validators` function, the threshold is never validated to be greater than 0.

This is contrast to the other spoke connection implementations, such as the EVM implementation in ConnectionV3::updateValidators:

```
require(_threshold > 0, "Invalid threshold");
```

While this may allow for easier testing, if by mistake such a configuration is left in production, it would allow arbitrary message approvals which would drain the spoke.

Impact

An incorrect threshold configuration of 0 in production would allow arbitrary message approvals which would drain the spoke.

Code Snippet

connection-v3.clar#L38-L47

Tool Used

Manual Review

Recommendation

Ensure that the threshold is greater than 0 in the `connection-v3::set-validators` function.

Discussion

gcranju

Acknowledged and Fixed https://github.com/icon-project/intent-relay/pull/366

abarbatei

Confirmed, the issue was fixed in the mentioned PR.

Issue L-5: Validators Are Not Deduplicated When Set [RESOLVED]

Source: https://github.com/sherlock-audit/2026-02-sodax-clarity-feb-9th/issues/18

Summary

When validators are set, there is no deduplication check. If, by mistake, duplicated keys are introduced, then the system risks not being able to sign any new messages.

Vulnerability Detail

In the `connection-v3::set-validators` function, the `validators-list` is never checked for duplicates.

If admin mistakenly passes `[A, A, B]` with threshold 3:

- Stored validators: `[compressed(A), compressed(A), compressed(B)]`
- Length check passes: `3 >= 3`
- But during verification, both copies of A recover the same key, so `index-of?` on `unique-validators-list` deduplicates them and only 1 is counted for A
- Effective unique validators: 2 (A and B), not 3
- System cannot validate any new message

Impact

System may reach a point where new messages cannot be validated until a new threshold is set or unique validators are set.

Code Snippet

intent-relay/contracts/stacks/contracts/connection/connection-v3.clar#L38-L47

Tool Used

Manual Review

Recommendation

In the `connection-v3::set-validators` function, check that the `validators-list` contains no duplicates and revert execution if it does.

Discussion

gcranju

Acknowledged and Fixed https://github.com/icon-project/intent-relay/pull/368

abarbatei

Confirmed, the issue was fixed in the mentioned PR.

The team opted to use a different approach where instead of reverting if duplicates are passed, the `validators-list` is filtered for unique elements, which are then processed by the setter.

Issue L-6: Public Key Compression Missing `0x04` Prefix Validation [RESOLVED]

Source: https://github.com/sherlock-audit/2026-02-sodax-clarity-feb-9th/issues/19

Summary

The `connection-v3::compress-public-key` function does not validate that the first byte of the input is `0x04` (the SEC1 uncompressed public key marker). Byte 0 is completely ignored during compression, so any 65-byte input with a valid x-coordinate body will silently produce a compressed key, even if the input is malformed.

Vulnerability Detail

The `connection-v3::compress-public-key` function accepts a list of non-compressed public keys which represent validators. It generates the compressed version by correctly taking the x-coordinate part of the public key

and extracting the y-parity element

then deriving the prefix and generating the final concatenation

However, nowhere in the function does it check that the signature starts with the `0x04` byte, which is the standard for uncompressed public keys.

If the admin passes keys with an incorrect prefix byte (e.g., `0x02`, `0x03`, `0xFF`, or `0x00` instead of `0x04`), the function silently accepts them and produces a compressed key based on whatever bytes happen to be at positions 1-32 and 64.

For keys that are genuinely uncompressed but simply have the wrong first byte, the output is still correct.

For inputs that are fundamentally not uncompressed keys (e.g. already-compressed keys padded to 65 bytes), the output is silently wrong and will never match any secp256k1-recover? result, effectively bricking that validator slot.

Impact

If admin accidentally passes malformed keys, those validators can never be matched during signature verification, requiring admin intervention to update the validator list.

Code Snippet

connection-v3.clar#L184-L190

Tool Used

Manual Review

Recommendation

Modify the key processing logic in the `connection-v3` contract to verify the prefix of the keys while compressing.

Discussion

gcranju

Acknowledged and Fixed https://github.com/icon-project/intent-relay/pull/368

abarbatei

Confirmed, the issue was fixed in the mentioned PR.

Issue L-7: Multiple Concatenations Can Be Chained More Simply [RESOLVED]

Source: https://github.com/sherlock-audit/2026-02-sodax-clarity-feb-9th/issues/20

Summary

When concatenating multiple buffs in clarity, the current `connection-v3::get-message-hash` uses `concat` in a way that makes further debugging and extensions harder to do.

This can be improved using a known Clarity pattern.

Vulnerability Detail

The `connection-v3::get-message-hash` function converts all needed inputs to buffers and then concatenates them, in order to calculate the `keccak256` hash.

The code is written with the `concat` keyword first, and ending inputs `(concat (concat (concat ... input1) input2) input3))`:

This makes further code extension significantly harder and can be improved by using a clearer pattern.

Impact

Increased code maintenance and extension costs.

Code Snippet

connection-v3.clar#L93-L102

Tool Used

Manual Review

Recommendation

Rewrite the `connection-v3::get-message-hash` function to use a `concat + parameter` first pattern, similar to how it is done here.

Example implementation:

```
(define-private (get-message-hash (src-chain-id uint) (src-address (buff 256))
↪  (dst-chain-id uint) (dst-address (buff 256)) (nonce uint) (payload (buff 4096)))
  (keccak256
    (concat (uint-to-32-bytes nonce)
    (concat (uint-to-32-bytes src-chain-id)
    (concat (uint-to-32-bytes dst-chain-id)
    (concat src-address
    (concat dst-address payload)))))
  )
)
```

Discussion

gcranju

Acknowledged and Fixed: https://github.com/icon-project/intent-relay/pull/367

abarbatei

Confirmed, the issue was fixed in the mentioned PR.

Issue L-8: Setting Rate Limit Does Not Immediately Apply New Max Availability Cap For External View [ACKNOWLEDGED]

Source: https://github.com/sherlock-audit/2026-02-sodax-clarity-feb-9th/issues/27

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

When the `rate-limit-set-action` command is sent by the hub, the current implementation calculates the remaining available withdraw balance and incorrectly caps it only to the old maximum amount before saving it to storage.

In case the new max amount is greater than the old one, this is not an issue, however, if the new maximum amount is less than the old one, after the set, the available amount is higher than the maximum itself that breaks the availability invariant in terms of data.

External integrators which read the information via `rate-limit-state::get-rate-limit` would see an erroneous, incorrect, value.

Vulnerability Detail

In the `rate-limit-impl::process-hub-message` function, when the `rate-limit-set-action` command is processed, the new rate limit information is extracted (`rate` and `max`):

and the available, updated with the existing values, is calculated:

```
(available (if (is-eq (get rate-per-second current-config) u0)
  max
  (compute-available token current-config)
))
```

However, the calculated `available` amount is never capped to the new `max` value, meaning that, if the new max value is less than the old max value, anyone relying externally on `rate-limit-state::get-rate-limit` would see an erroneous value.

Note: on chain execution will always use the correct value, since, after each withdraw, the available amount is recalculated using the correct max and committed to storage.

Only in between setting a new amount and the first withdraw, will the `rate-limit-state::get-rate-limit` show an incorrect amount.

Impact

Only in between setting a new amount and the first withdraw, the `rate-limit-state::get-rate-limit` may show an incorrect amount, higher than the true value, if the previous maximum allowed available amount was higher than the newer one. External integrators relying on this data may make incorrect decisions based on inflated availability.

Code Snippet

rate-limit-impl.clar#L323-L326

Tool Used

Manual Review

Recommendation

In the `rate-limit-impl::process-hub-message` function, when the `rate-limit-set-action` command is processed, cap the resulting `available` amount to the `max` variable before committing it to storage.

Discussion

gcranju

This behavior is understood. When updating the rate limit, we intentionally preserve the accumulated available amount based on the previous configuration. If the new max is lower than the previous one, the effective cap is still enforced during verifyWithdraw, so on-chain execution cannot exceed the new maximum.

Additionally, we do not rely externally on the available value returned by rate-limit-state::get-rate-limit. Any availability checks use the canonical get-available logic in rate-limit-impl, which always applies the correct max and rate configuration.

abarbatei

Understood that Sodax itself does not use `rate-limit-state::get-rate-limit` but any potential 3rd party that uses it, as mentioned in the issue, will see in that particular case, an incorrect value. Thus, the issue itself has value to remain as a note/info, even if not resolved.

Issue L-9: Avoid Using Unwrap Panic [RESOLVED]

Source: https://github.com/sherlock-audit/2026-02-sodax-clarity-feb-9th/issues/28

Summary

Throughout the codebase there are multiple instances where unwrap-panic is used.

Using `unwrap-panic` is discouraged as it both increases debug difficulty in case of an error while also making it harder for external integrators to use, since a panic revert would end the transactions and not allow 3rd parties to handle a specific error code.

Vulnerability Detail

When `unwrap-panic` is called on an `(err ...)` or `(none)` value, it reverts the current transaction fully:

> unwrap throws a runtime error, aborting any further processing of the current transaction.

There are multiple instances in the codebase where `unwrap-panic` is used.

Impact

Increased difficulty for external integrators.

Code Snippet

All contracts, some examples:

rate-limit-impl.clar#L178

rate-limit-impl.clar#L184

rate-limit-impl.clar#L76-L79

Tool Used

Manual Review

Recommendation

Wherever possible, change `unwrap-panic` to `unwrap!` plus a separate error code.

Also, in several instances, a `try!` can be used, example when using `unwrap-panic` on the result from calling external functions.

Example:

```
 (define-public (pause (payload (buff 4096)) (connection <connection-contract>))
   (let (
-        (data (unwrap-panic (decode-and-validate-payload payload connection)))
+        (data (try! (decode-and-validate-payload payload connection)))
         (message (get data data))
         (kind (get kind data))
       )
       ;; Verify the signature
       (asserts! (is-eq kind pause-action) err-invalid-action)
-      (unwrap-panic (contract-call? .rate-limit-state pause))
+      (try! (contract-call? .rate-limit-state pause))
       (ok true)
     )
 )
```

Discussion

gcranju

Acknowledged and Replace with try! where feasible: https://github.com/icon-project/sodax-contracts/pull/599

abarbatei

Confirmed, the issue was fixed in the mentioned PR.

Issue L-10: Overlapping Error Code Ranges [RESOLVED]

Source: https://github.com/sherlock-audit/2026-02-sodax-clarity-feb-9th/issues/29

Summary

Throughout the codebase, multiple contracts define error codes in the same `100-199` range, causing overlaps across contracts that call each other. This makes errors indistinguishable in cross-contract call chains.

Vulnerability Detail

The following contracts all define error codes starting at `u100`:

- `asset-manager-impl`: u100-u111
- `asset-manager-state`: u100-u101
- `rate-limit-state`: u100-u102
- `intent-filler-state`: u100-u101
- `connection-v3`: u100-u110
- `connection-v3-state`: u100
- `soda`: u100-u101
- `bnusd`: u100-u101

For example, `u100` means:

- `err-not-admin` in `asset-manager-impl`
- `err-not-asset-manager` in `asset-manager-state`
- `err-not-rate-limit` in `rate-limit-state`
- `err-not-admin` in `connection-v3`.

These contracts directly call each other (asset-manager → rate-limit-state, asset-manager → connection-v3), so when a transaction reverts with `u100`, the source may be ambiguous.

Exceptions are the contracts `rate-limit-impl` (`900-913`) and `intent-filler-impl` (`800-813`) which use distinct ranges.

Impact

Error codes are ambiguous in cross-contract call chains, making debugging and error handling unreliable.

Code Snippet

- asset-manager-impl.clar#L9-L20
- asset-manager-state.clar#L10-L11
- rate-limit-state.clar#L11-L13
- intent-filler-state.clar#L10-L11
- connection-v3.clar#L10-L15
- connection-v3-state.clar#L8
- soda.clar#L3
- bnusd.clar#L3

Tool Used

Manual Review

Recommendation

Assign each contract a unique error code range. Use ranges above 999 to avoid any potential collision with other integrating protocols. Examples:

- `asset-manager-impl`: 1000+
- `asset-manager-state`: 2000+
- `rate-limit-state`: 3000+
- `intent-filler-state`: 4000+
- `connection-v3`: 5000+
- `connection-v3-state`: 6000+
- `soda`: 7000+
- `bnusd`: 8000+

Note, for the token contracts, the `err-not-token-owner` error must remain 4 in order to keep SIP-10 compliance, do not change that.

Discussion

gcranju

Acknowledged and Fixed: https://github.com/icon-project/sodax-contracts/pull/598

abarbatei

Confirmed, the issue was fixed in the mentioned PR.

Issue L-11: Severe Lack Of Event Printing [RESOLVED]

Source: https://github.com/sherlock-audit/2026-02-sodax-clarity-feb-9th/issues/30

Summary

The Clarity `asset-manager-impl` and `rate-limit-impl` contracts emit no `print` events for any state-changing operations. The EVM equivalents (`SpokeAssetManager.sol`, `RateLimit.sol`) emit events for all significant state changes. This breaks off-chain observability and indexing parity with other spokes.

Vulnerability Detail

In Clarity, the print command is the equivalent of Solidity's `emit`. The EVM spoke contracts emit events for configuration changes, deposits, withdrawals, rate limit updates, admin rotations, signer updates, pauses, and resets. The Clarity contracts perform the same operations but without emitting any events.

EVM events with no Clarity equivalent

- `SpokeAssetManager.sol`
  - `emit RateLimitUpdated(oldRateLimit, newRateLimit)` when rate limit contract is changed
  - `emit ConnectionUpdated(oldConnection, newConnection)` when connection contract is changed
- `RateLimit.sol`
  - `emit TokenConfigUpdated(token, ratePerSecond, maxAvailableWithdrawal)` when a token's rate limit is set
  - `emit RateLimitReset(token)` when a token's rate limit is reset
  - `emit HubAdminUpdated(hubAdmin)` when hub admin is changed
  - `emit HubSignersUpdated(signers)` when hub signers are rotated

For Clarity, the `asset-manager-impl` and `rate-limit-impl` contracts print no events across all state-changing functions.

Impact

No off-chain observability for deposits, withdrawals, configuration changes, admin rotations, signer updates, rate limit changes, pauses, or resets on the Stacks spoke. Indexers, dashboards, and monitoring systems cannot track contract activity, breaking operational parity with the EVM and other spokes.

Code Snippet

- asset-manager-impl.clar#L36-L130
- rate-limit-impl.clar#L66-L187

Tool Used

Manual Review

Recommendation

Add print events to all public state-changing functions.

A general print/event layout structure can also be used for ease of off-chain processing. Example of such a structure:

At a minimum, add `print` events to all state-changing functions in `asset-manager-impl.clar` and `rate-limit-impl.clar` to match the EVM event coverage, emitting events for: deposits, withdrawals, rate limit configuration, rate limit resets, hub admin changes, signer rotations, pauses, and admin/connection/contract upgrades.

Discussion

gcranju

Acknowledged and Fixed https://github.com/icon-project/sodax-contracts/pull/597

abarbatei

Confirmed, the issue was fixed in the mentioned PR.

While the team opted to implement only the minimum 1:1 events with the other spokes (6 events), further events would of been useful.

Issue L-12: Consecutive Initializations Are Possible [RESOLVED]

Source: https://github.com/sherlock-audit/2026-02-sodax-clarity-feb-9th/issues/31

Summary

The `init` functions in `asset-manager-impl`, `rate-limit-impl`, and `connections-v3` contracts do not validate that `hub-chain-id` is non-zero. Since `hub-chain-id == u0` is the initialization guard, passing `u0` allows `init` to be called repeatedly, overwriting all configuration each time.

Vulnerability Detail

All three contracts use the same initialization pattern:

The `init` function checks `is-not-initialized` (hub-chain-id must be `u0`) but never asserts that the provided `hub-chain-id` parameter is non-zero. If `init` is called with `hub-chain-id = u0`, the guard passes, all other parameters (connection, rate-limit, hub-asset-manager, hub-admin, hub-signers) are overwritten, and `hub-chain-id` remains `u0`, leaving the contract "uninitialized" and re-callable.

Impact

The admin can re-initialize the contract multiple times as long as he passes `hub-chain-id` as `u0`, overwriting the connection, rate-limit, and hub addresses. The one-time initialization invariant is not enforced.

Code Snippet

- asset-manager-impl.clar#L36-L46
- rate-limit-impl.clar#L66-L82
- connection-v3.clar#L24-L29

Tool Used

Manual Review

Recommendation

Check at the start of each `init` function that the hub chain ID is not 0.

Example implementation:

Discussion

gcranju

Acknowledged and Fixed https://github.com/icon-project/sodax-contracts/pull/601
And for connection fixed in https://github.com/icon-project/intent-relay/pull/369

abarbatei

Confirmed, the issue was fixed in the mentioned PR.

Issue L-13: Incorrect Zero Amount Check In Deposits [RESOLVED]

Source: https://github.com/sherlock-audit/2026-02-sodax-clarity-feb-9th/issues/33

Summary

The `transfer` function in `asset-manager-impl` contract allows 0 amount transfers due to using >= instead of > in the amount check.

Vulnerability Detail

The assertion:

```
(asserts! (>= amount u0) err-amount-invalid)
```

passes when `amount` is `u0` since `0 >= 0` is true. The transaction still reverts later within the `asset-manager-state::deposit` function, when either SIP-10::transfer or stx-transfer? is called, which reject zero amounts, but the revert comes as an `unwrap-panic` runtime abort instead of the intended `err-amount-invalid` error.

Impact

Zero-amount transfers panic-abort instead of returning a clean error code, making the failure reason opaque to integrators and off-chain tools.

Code Snippet

asset-manager-impl.clar#L99

Tool Used

Manual Review

Recommendation

Change the check in the `asset-manager-impl::transfer` function to strictly greater than zero:

Discussion

gcranju

Acknowledged and Fixed https://github.com/icon-project/sodax-contracts/pull/600

abarbatei

Confirmed, the issue was fixed in the mentioned PR.

Issue L-14: Missing Initialization Checks Allow Signatureless Message Verification At Deployment [RESOLVED]

Source: https://github.com/sherlock-audit/2026-02-sodax-clarity-feb-9th/issues/34

Summary

In contracts that require an initialization first, no public function outside of `init` itself checks whether the contract has been initialized. Operations like message verification, token transfers, and deposits are callable in an uninitialized state with potentially dangerous default values.

Of relevance is the `connection-v3::verify-message` function, which accepts any message with zero valid signatures when `validators-threshold` defaults to `u0`.

Vulnerability Detail

All state variables are set to zero/empty values at deployment.

The `init` function sets critical configuration but no other function checks `is-initialized` before executing. Additionally, in `connection-v3`, `init` only sets `chain-id`. Validators and threshold are set separately via `set-validators`, creating an additional window even after `init`.

Per relevant contract we can see that:

`connection-v3`: `validators-threshold` defaults to `u0`, `validators` defaults to empty list:

The `verify-message` function checks `(>= (len signatures) threshold)` which is true for an uninitialized contract. Then `verify-signatures` returns true, same logic. Any message passes with zero valid signatures.

This is exploitable in the window between deployment and `set-validators`, or if `init` is called before validators are configured.

A separate issue is that `send-message` may also send messages with `src-chain-id = u0` in this case, which may confuse off-chain monitors.

`asset-manager-impl`: `hub-asset-manager` defaults to `none`:

- `transfer`: `unwrap-panic` on `get-hub-asset-manager` (returns `none`) causes a runtime abort instead of a clean error
- `recv-message`: same `unwrap-panic` abort on `none` hub-asset-manager

Impact

Before `init` and `set-validators` are called, `connection-v3::verify-message` accepts any message without signature verification. Any contract already pointing to this connection

(e.g. if asset-manager is initialized before connection validators are set) would process unauthenticated cross-chain messages, allowing unauthorized withdrawals.

Code Snippet

- connection-v3.clar#L72-L83
- connection-v3.clar#L64-L70
- asset-manager-impl.clar#L97-L117
- asset-manager-impl.clar#L130-L159
- rate-limit-impl.clar#L94-L187

Tool Used

Manual Review

Recommendation

Add an `is-initialized` check (assert `hub-chain-id != u0` or equivalent) to all non-admin public functions in all contracts that have an `init` function (`connection-v3`, `asset-manager-impl` and `rate-limit-impl`).

Discussion

gcranju

I think the other points shouldn't be a big issue given our deployment flow. This one was concerning though, so I checked it and added the guard to prevent verification when the threshold is zero. http://github.com/icon-project/intent-relay/pull/366/changes allowbreak #diff-6e3291c657f007f4601736e0d3f515288637e9a4b7cd0ba89618e00d12f82f34R73

abarbatei

Alright, confirming the fix looks good and mitigates the most impactful problem within this finding.

Issue L-15: Oversized Buffer Allocations [RESOLVED]

Source: https://github.com/sherlock-audit/2026-02-sodax-clarity-feb-9th/issues/36

Summary

Throughout the codebase, there are multiple state variables and function parameters that use buffer sizes far exceeding what is needed (e.g. `(buff 4096)` for 33-byte compressed public keys), wasting storage and allowing malformed data to be stored without detection.

Vulnerability Detail

Throughout the codebase, buffer types are significantly oversized relative to their actual expected content:

`rate-limit-state`:

- `hub-admin`: `(buff 4096)` but stores a 33-byte compressed secp256k1 public key
- `hub-signers`: `(list 10 (buff 4096))` is same case, per signer
- `hub-manager`: `(buff 100)` but should be sized to the actual hub manager address format

`asset-manager-state`:

- `hub-asset-manager` is `(buff 100)` with no length validation on set

`connection-v3-state`:

- `validators` is `(list 50 (buff 100))` but validators are 33-byte compressed public keys

Oversized buffers increase storage costs and allow incorrectly formatted data to be silently accepted and stored.

Note: both `hub-asset-manager` and `hub-manager` variables store EVM contract addresses with the current implementation of the bridge (20 bytes via `abi.encodePacked(msg.sender)`).

Their size in practice is `(buff 20)`, however, since the hub manager may change and Sodax may migrate to other chains, to not require a state contract migration (which defeats the whole purpose of separating state from implementation), these two variables are actually better off to be increased to a `(buff 256)` size, similar to the `dst-address` / `src-address` parameters, which use `(buff 256)` in `connection-v3::send-message` and `rate-limit/asset-manager::recv-message`. This is because, in code, the hub manager is compared with `src-address (buff 256)` via `is-eq`.

Impact

Unnecessary storage overhead. Malformed or incorrectly sized data can be written to state without error, which may lead to downstream comparison failures.

Code Snippet

- rate-limit-state.clar#L27-L29
- asset-manager-state.clar#L22
- connection-v3-state.clar#L16

Tool Used

Manual Review

Recommendation

Modify the size of all buffer types to match their actual content:

- Compressed public keys/Validator keys: `(buff 33)`
- Hub manager / hub asset-manager addresses: `(buff 256)` (although EVM contract addresses are stored, allowing 256 allows support for different hub address format in case of hub migration)

Add length assertions wherever these values are set to reject malformed inputs early.

Concretely, the following variables should be changed as such:

Variable	Current	Should be
`rate-limit-state::hub-admin`	`(buff 4096)`	`(buff 33)`
`rate-limit-state::hub-signers`	`(list 10 (buff 4096))`	`(list 10 (buff 33))`
`rate-limit-state::hub-manager`	`(buff 100)`	`(buff 256)`
`asset-manager-state::hub-asset-manager`	`(buff 100)`	`(buff 256)`
`connection-v3-state::validators`	`(list 50 (buff 100))`	`(list 50 (buff 33))`

Discussion

gcranju

Acknowledged and Fixed

https://github.com/icon-project/sodax-contracts/commit/a6d92c3f9cd0ab3dec8f505da5e88de305cbc7e6 and I think for validators, there might be some protocol changes so, I think 100 would be fine.

abarbatei

Confirmed, the issue was fixed in the mentioned PR.

Issue L-16: Miscellaneous QA Issues in Asset Manager [RESOLVED]

Source: https://github.com/sherlock-audit/2026-02-sodax-clarity-feb-9th/issues/37

Summary

Collection of low-severity code quality and correctness issues found in the `asset-manager-impl` and `asset-manager-state` contracts.

Vulnerability Detail

The following is a list of findings that are relevant to be resolved in order to improve codebase code quality and correctness:

1. `get-token-balance` is `define-public` but should be `define-read-only`

The `asset-manager-state::get-token-balance` only reads state and returns a balance. Being `define-public` means it costs a transaction fee to call externally and can mutate state in theory. It should be `define-read-only`.

- location: asset-manager-state.clar#L86-L94

2. Dead code: `sender` variable in `recv-message` is unused

In `asset-manager-impl::recv-message`, the decoded sender field is bound but never referenced.

- location: asset-manager-impl.clar#L146

3. `transfer` does not emit a print event or return the connection sequence number

The `asset-manager-impl::transfer` function completes a cross-chain deposit but emits no `print` event and does not return the connection sequence number (`sn`), making it harder for off-chain indexers to track deposits.

- location: asset-manager-impl.clar#L97-L117

4. No validation of RLP element count in `decode-transfer-raw`

`asset-manager-impl::decode-transfer-raw` extracts exactly 5 elements from the RLP list but does not assert the list has exactly 5 entries. Extra trailing elements would be silently ignored and may indicate the presence of corrupt/unwanted messages.

- location: asset-manager-impl.clar#L223-L233

5. Insufficient balance uses same error code as invalid amount

In `asset-manager-impl::recv-message`, `(asserts! (and (> amount u0) (>= token-balance amount)) err-amount-invalid)` conflates zero-amount and insufficient-balance failures into a single error, making debugging harder.

- location: asset-manager-impl.clar#L152

6. Misleading variable name `token-trait` in `recv-message`

In `asset-manager-impl::recv-message`, the variable `token-trait` actually holds a `principal` (from `contract-of`), not a trait reference. This is misleading and reduces code clarity.

- location: asset-manager-impl.clar#L149

7. Redundant wrapping in `asset-manager-state` passthrough functions

`asset-manager-state::verify-message` wraps the call to `connection::verify-message` in a redundant `(match ... value (ok value) err-code (err err-code))` instead of directly returning the result. Similarly, `asset-manager-state::send-message` wraps its call in `(ok (unwrap-panic ...))` instead of directly returning it. Since both are passthroughs, the extra wrapping is unnecessary.

- location: asset-manager-state.clar#L100
- location: asset-manager-state.clar#L106

Code Snippet

- asset-manager-state.clar#L86-L94
- asset-manager-impl.clar#L146
- asset-manager-impl.clar#L97-L117
- asset-manager-impl.clar#L223-L233
- asset-manager-impl.clar#L152
- asset-manager-impl.clar#L149
- asset-manager-state.clar#L100
- asset-manager-state.clar#L106

Tool Used

Manual Review

Recommendation

Apply the following fixes, corresponding to each of the issues above:

1. Change `asset-manager-state::get-token-balance` from `define-public` to `define-read-only`
2. Remove the unused `sender` binding from the `asset-manager-impl::recv-message` function, or add it (with other relevant information) in a `print` statement

3. Add `print` event and return `sn` from the `asset-manager-impl::transfer` function
4. Validate RLP list length in `asset-manager-impl::decode-transfer-raw` to be exactly 5
5. Use separate error codes for zero-amount vs insufficient-balance in the `asset-manager-impl::recv-message` function
6. Rename `token-trait` to `token-principal` (or similar) in `asset-manager-impl::recv-message`
7. Simplify `asset-manager-state::verify-message` and `asset-manager-state::send-message` to directly return the last connection calls instead of re-wrapping with `match /(ok (unwrap-panic ...))`

Discussion

gcranju

Acknowledged

1. Since get-toke-balance is fetching balance of token interacting with token contract, that doesn't allow method to be public
2. We will be tracking offline with connection event and fetching transaction, so not planning to add event in transfer as of now Others are fixed in #28 in this specific comment https://github.com/icon-project/sodax-contracts/commit/3a75cdd6ec1e75359119e5ab54a8ea4618b7cccc

abarbatei

Understood for the mentioned points. For point (5), the idea was to split:

into two separate asserts, for easier debugging:

If this is relevant for you please fix it, otherwise will consider that sub-finding also acknowledged.

Confirming the other sub-findings are fixed correctly.

gcranju

Oh, yeah fixed this https://github.com/icon-project/sodax-contracts/pull/599/changes/5f0287733d3245413ae8317aadbc59b20a706b8d

abarbatei

Super, thanks! Confirming that the last sub-issue was fixed in the mentioned commit.

Issue L-17: Old-Style Tuple Syntax Used Instead of Modern Curly-Brace Syntax [RESOLVED]

Source: https://github.com/sherlock-audit/2026-02-sodax-clarity-feb-9th/issues/38

Summary

Multiple contracts use the legacy `(tuple (key value) ...)` syntax instead of the modern `{ key: value, ... }` syntax introduced in Clarity 2.

Vulnerability Detail

The legacy tuple syntax is harder to read and not idiomatic for current Clarity versions. There are several such instances of old style tuples being used in the following contracts: `asset-manager-impl`, `asset-manager-state` and `rate-limit-state`.

Note: exact instances are listed in the Code Snippet section

Impact

No functional impact. Reduces code readability and consistency with modern Clarity conventions.

Code Snippet

`asset-manager-impl`:

- L102: `transfer` message construction
- L174: `recv-message` token-trait binding
- L203: `encode-transfer` parameter type
- L231: `decode-transfer` return value
- L244: `decode-transfer-raw` return value

`asset-manager-state`:

- L125: `get-impl-params`

`rate-limit-state`:

- L176: `get-rate-limit` default value
- L181: `get-impl-params`

Tool Used

Manual Review

Recommendation

Replace all `(tuple (key value) ...)` instances with `{ key: value, ... }` syntax throughout the codebase.

Discussion

gcranju

Acknowledged and Fixed in #28 in the following specific comment https://github.com/icon-project/sodax-contracts/commit/64604131eef986f7f983160dcd367c69d1bf1e95

abarbatei

Confirmed, the issue was fixed in the mentioned PR.

Issue L-18: Miscellaneous QA Issues in Rate Limiter [RESOLVED]

Source: https://github.com/sherlock-audit/2026-02-sodax-clarity-feb-9th/issues/39

Summary

Collection of low-severity code quality and correctness issues found in the `rate-limit-impl` and `rate-limit-state` contracts.

Vulnerability Detail

The following is a list of findings that are relevant to be resolved in order to improve codebase code quality and correctness:

1. Misleading comment: "Verify the signature" in `recv-message`

The comment says "Verify the signature" but the function body just calls `process-hub-message`; signature verification already happened in `decode-and-validate-payload`.

- location: rate-limit-impl.clar#L167

2. Unused `token` parameter in `compute-available` and `get-available`

Both `rate-limit-impl::compute-available` and `rate-limit-impl::get-available` accept a `token` parameter that is never used in the function body.

- location: rate-limit-impl.clar#L272
- location: rate-limit-impl.clar#L221

3. `reset-limit` in `rate-limit-state` is never called

The `rate-limit-state::reset-limit` public function is defined but never invoked from any implementation contract or hub message handler.

- location: rate-limit-state.clar#L114-L119

4. `get-rate-limit` uses verbose match instead of `default-to`

`rate-limit-state::get-rate-limit` manually matches on `map-get?` and returns a default tuple. Could be simplified with `default-to`.

- location: rate-limit-state.clar#L174-L177

5. Redundant wrapping in `rate-limit-state::verify-message` passthrough

`rate-limit-state::verify-message` wraps the call to `connection::verify-message` in `(ok (unwrap-panic ...))` instead of directly returning the result. Since this is a passthrough, the extra wrapping is unnecessary.

- location: rate-limit-state.clar#L151

Code Snippet

- rate-limit-impl.clar#L167
- rate-limit-impl.clar#L272
- rate-limit-impl.clar#L221
- rate-limit-state.clar#L114-L119
- rate-limit-state.clar#L174-L177
- rate-limit-state.clar#L151

Tool Used

Manual Review

Recommendation

Apply the following fixes, corresponding to each of the issues above:

1. Fix misleading "Verify the signature" comment in `rate-limit-impl::recv-message`
2. Remove unused `token` parameters from the `rate-limit-impl::compute-available` and `rate-limit-impl::get-available` functions
3. Remove the `rate-limit-state::reset-limit` function
4. Use `default-to` in `rate-limit-state::get-rate-limit`
5. Simplify `rate-limit-state::verify-message` to directly return the connection call instead of wrapping with `(ok (unwrap-panic ...))`

Discussion

gcranju

Acknowledged 2. The token parameter is intentionally unused: it preserves a read-only interface, and avoids pulling config from contract state, with the token passed explicitly for clarity. Others Fixed in #28 explicitly in commit https://github.com/icon-project/sodax-contracts/commit/68cc432c7752be0226f45be15e3b4168ecfa06f6

abarbatei

Understood for point (2). Confirming the other are fixed correctly.

Issue L-19: Miscellaneous QA Issues in Connection Component [RESOLVED]

Source: https://github.com/sherlock-audit/2026-02-sodax-clarity-feb-9th/issues/40

Summary

Collection of low-severity code quality and correctness issues found in the `connection-v3` and `connection-v3-state` contracts.

Vulnerability Detail

The following is a list of findings that are relevant to be resolved in order to improve codebase code quality and correctness:

1. `set-receipt` private wrapper is unused

The `connection-v3::set-receipt` private function wraps `connection-v3-state::set-receipt` but is never called; `verify-message` calls the state contract directly via `contract-call?`.

- location: connection-v3.clar#L172-L174

2. Redundant `begin` wrapping `let` in `verify-signatures`

`connection-v3::verify-signatures` wraps a `let` expression in a `begin` block. Since `let` is already a single expression, the `begin` is unnecessary.

- location: connection-v3.clar#L164

3. Typo in state variable name: `chain-nid` instead of `chain-id`

`connection-v3-state` stores the chain ID in a variable called `chain-nid` and the setter references `chain-id-in`, creating a naming inconsistency.

- location: connection-v3-state.clar#L15

4. `recover-public-key` is `define-public` but should be `define-read-only`

`connection-v3::recover-public-key` performs no state mutations, it only calls `secp256k1-recover?` which is a pure function. It should be `define-read-only`.

- location: connection-v3.clar#L114-L131

5. Unused `rs` variable in `recover-public-key`

In `connection-v3::recover-public-key`, the variable `rs` is bound via `(slice? signature u0 u64)` but never referenced in the function body.

- location: connection-v3.clar#L117

Code Snippet

- connection-v3.clar#L172-L174
- connection-v3.clar#L164
- connection-v3-state.clar#L15
- connection-v3.clar#L114-L131
- connection-v3.clar#L117

Tool Used

Manual Review

Recommendation

Apply the following fixes, corresponding to each of the issues above:

1. Remove the dead `connection-v3::set-receipt` private wrapper, or use it
2. Remove the redundant `begin` block in `connection-v3::verify-signatures`
3. Rename `chain-nid` to `chain-id` in `connection-v3-state` for consistency
4. Change `connection-v3::recover-public-key` from `define-public` to `define-read-only`
5. Remove the unused `rs` variable from `connection-v3::recover-public-key`

Discussion

gcranju

3. We planned to use chain-id, but chain-id is reserved word in clarity so, chose chain-nid Other fixed here, https://github.com/icon-project/intent-relay/pull/370

abarbatei

Correct for point (3), we overlooked that. Confirming the other points are fixed correctly.

Issue L-20: Add Standard Checks For All Saved Principals [RESOLVED]

Source: https://github.com/sherlock-audit/2026-02-sodax-clarity-feb-9th/issues/41

Summary

Throughout the codebase there are sensitive principals that are saved in the storage contracts. None of these principals are checked to be standard (meaning to belong to the current network). Testnet principals can be set as admin, implementation, connection, or rate-limit addresses on mainnet (and vice versa).

Vulnerability Detail

Clarity allows testnet-format principals to be passed and stored on mainnet without error. By using the is-standard function, a check can be done to ensure no non-standard principals can be added.

Impact

By mistake, a testnet principal may be used instead of a mainnet principal, rendering key functionality unusable.

Code Snippet

- asset-manager-impl.clar#L36-L85
- asset-manager-state.clar#L28-L46
- rate-limit-impl.clar#L66-L77
- rate-limit-state.clar#L44-L63
- connection-v3.clar#L31-L37
- connection-v3-state.clar#L25-L31
- bnusd.clar#L19

Tool Used

Manual Review

Recommendation

Add an `is-standard` check before every `var-set` or `map-set` that stores a principal. At minimum, guard all `set-admin`, `set-*-impl`, `init` and `transfer-contract-ownership` functions.

Discussion

gcranju

https://github.com/icon-project/intent-relay/pull/371 For others, fixed in same pr #28 in specific commit https://github.com/icon-project/sodax-contracts/commit/904946173fb316b0feba8c7e9e6f6325691ceba6

abarbatei

Confirmed, the issue was fixed but the full fix is split in the mentioned PR and in the pull/602 fix.

Issue L-21: Single-Step Admin Transfer With No Recovery Mechanism [RESOLVED]

Source: https://github.com/sherlock-audit/2026-02-sodax-clarity-feb-9th/issues/42

Summary

`connection-v3::set-admin` and `bnusd::transfer-contract-ownership` immediately overwrite the admin/owner to a new address in a single step. If the wrong address is provided, admin access is permanently lost with no recovery path.

Vulnerability Detail

Both `connection-v3::set-admin` and `bnusd::transfer-contract-ownership` use a single-step admin transfer pattern where the new admin is set immediately upon calling. There is no pending state or acceptance step, meaning a typo or incorrect address results in irreversible loss of admin privileges.

A two-step transfer pattern (propose + accept) is the standard mitigation: the current admin proposes a new admin, and the new admin must call an `accept` function to finalize the transfer. This ensures the new address is valid and controlled by the intended party.

Impact

Permanent loss of admin access if the wrong address is provided, with no way to recover.

Code Snippet

- connection-v3.clar#L32-L36
- bnusd.clar#L24

Tool Used

Manual Review

Recommendation

Implement a two-step admin transfer pattern (propose + accept) in the `connection-v3` and `bnusd` contracts.

Note: this is equivalent to implementing OpenZeppelin's Ownable2Step functionality, but in Clarity.

Discussion

gcranju

Acknowledged and Fixed

https://github.com/icon-project/sodax-contracts/pull/602
https://github.com/icon-project/intent-relay/pull/371

abarbatei

Confirming the issue is fixed as it is in the mentioned commits, but a slight observation:

In the `connection` component the accept/propose entry point is in the implementation contract while in the `rate-limit` and `asset-manager` components, the entry points for accept/propose are in the state contract. The difference has no impact on functionality, only a slight cost increase when executing these commands and an implementation deviation. If this is not indented, change it and ping us, otherwise leave it as so.

Disclaimers

Sherlock does not provide guarantees nor warranties relating to the security of the project.

Usage of all smart contract software is at the respective users' sole risk and is the users' responsibility.